

DeskFlow Documentation

Internal IT Service Request Portal — Full-Stack Technical Manual

github.com/AndileP3/Deskflow

Tech Stack: React, Node.js, Express, MongoDB, JWT, Swagger/OpenAPI | **Author:** AndileP3 | **Date:** July 2026

Table of Contents

- 1. Project Overview & Repository Structure
- 2. Prerequisites & Environment Setup
- 3. Step-by-Step Installation & Running
- 4. Architecture & Security Mechanics
- 5. API Reference & Swagger Specifications
- 6. 5-Day Commit Plan (Git Hygiene)
- 7. Troubleshooting & Common Pitfalls
- 8. Next Steps & Post-MVP Roadmap

1. Project Overview & Repository Structure

DeskFlow is an internal IT service request portal designed to streamline ticket submission for employees and ticket lifecycle resolution for system administrators. Built as a full-stack JavaScript application, it utilizes standard RESTful patterns, JWT authentication, and role-based authorization guards.

Repository Directory Layout

```
deskflow/
├── backend/                                # Node.js + Express API
│   ├── config/
│   │   └── db.js                          # MongoDB connection setup
│   ├── controllers/
│   │   ├── authController.js             # Login logic & JWT signing
│   │   └── ticketController.js          # CRUD operations for tickets
│   ├── middleware/
│   │   ├── auth.js                      # protect() + authorize() role guards
│   │   └── errorHandler.js             # Centralized error & 404 handler
│   ├── models/
│   │   ├── User.js                      # User schema & bcrypt hashing
│   │   └── Ticket.js                   # Ticket schema with status/priority enums
│   ├── routes/
│   │   ├── authRoutes.js               # POST /api/auth/login
│   │   └── ticketRoutes.js             # /api/tickets endpoints
│   ├── swagger.js                      # OpenAPI spec generation
│   ├── seed.js                         # Demo data seeder script
│   └── server.js                       # Express app entrypoint
├── frontend/                             # React SPA
│   ├── src/
│   │   ├── api/api.js                  # Axios client & request interceptors
│   │   ├── context/AuthContext.js      # Global state via React Context
│   │   ├── components/                 # Login, Navbar, Dashboards & Ticket views
│   │   └── App.js                      # Client routing & view management
├── postman/
│   └── DeskFlow.postman_collection.json # API collection for testing
└── README.md
```

2. Prerequisites & Environment Setup

Tool	Required Version	Verification Command
Node.js	18+	<code>node -v</code>
npm	9+	<code>npm -v</code>
MongoDB	Local install or Atlas Cluster	<code>mongod --version</code>
Git	Recent release	<code>git --version</code>
Postman	Latest (optional)	—

3. Step-by-Step Quickstart

1. Clone & Git Init

```
git init
git add .
git commit -m "chore: initial project scaffold"
git remote add origin https://github.com/AndileP3/Deskflow.git
git branch -M main
git push -u origin main
```

2. Backend Setup

```
cd backend
npm install
cp .env.example .env

# Configure .env:
# PORT=5000
# MONGO_URI=mongodb://127.0.0.1:27017/deskflow
# JWT_SECRET=your_32_character_random_string
# JWT_EXPIRES_IN=8h

# Seed Demo Users & Sample Tickets
npm run seed

# Launch Development Server
npm run dev
```

Default Demo Credentials

Employee: `employee@deskflow.com` / `password123`

Admin: `admin@deskflow.com` / `password123`

3. Frontend Setup

```
cd ../frontend
npm install
cp .env.example .env

# Configure frontend/.env:
# REACT_APP_API_URL=http://localhost:5000/api

npm start
```

4. Architecture & Security Mechanics



Security Core Concepts

- **Password Hashing:** Passwords are hashed using `bcrypt` via a pre-save hook on the Mongoose `User.js` model before storage.
- **Role Authorization:** Server-side role validation using `authorize('Admin')` middleware prevents non-admin users from updating ticket statuses (returns HTTP 403).
- **Data Scoping:** Employees query `{ createdBy: req.user._id }` to see only their tickets, while Admins view all tickets populated with employee details.
- **Input Validation:** Request payload bodies are validated with `express-validator` to prevent malformed data insertion.

5. API Reference

Method	Route	Access Level	Description
POST	/api/auth/login	Public	Authenticates user and returns JWT + profile
POST	/api/tickets	Employee	Creates a new service ticket (title, description, priority)
GET	/api/tickets	Employee / Admin	Returns scoped tickets list (supports <code>?status=</code> filter)
PUT	/api/tickets/:id	Admin	Updates ticket status (Open, In Progress, Resolved)
GET	/api/health	Public	API status health check

6. 5-Day Commit Plan (Git Hygiene)

Day 1–2: Backend Core • Initialize Express & MongoDB connection • Add User and Ticket Mongoose schemas • Implement login & JWT signing • Add auth middleware & error handling • Configure Swagger docs & test via Postman	Day 3–5: Frontend & Polish • Bootstrap React app & AuthContext • Build Login & Dashboard shell components • Connect TicketForm & TicketList to API • Implement Admin status updates • Polish UI & finalize documentation
---	--

7. Troubleshooting Guide

Symptom	Likely Cause	Resolution
ECONNREFUSED	MongoDB service not running	Run <code>mongod</code> locally or verify Atlas connection string
401 Unauthorized	Missing or expired JWT	Re-login to issue new token in <code>Authorization</code> header
403 Forbidden	Role permission mismatch	Ensure account has <code>Admin</code> role when calling PUT endpoints
CORS error	Missing CORS middleware	Verify <code>app.use(cors())</code> in <code>server.js</code>

8. Next Steps & Post-MVP Roadmap

- Audit trail for ticket status updates and admin comments
- Pagination support on `GET /api/tickets`
- User self-registration and password recovery workflows
- Attachment upload handling for screenshots/logs
- Automated email alerts on status changes